



ARTI 502

Deep Learning: 1st Assignment

Training-Testing Neural Net with MNIST Dataset using Pytorch

Year 2026 Term 1

Group 1 – 9FA01S

Group Members:

#	Member Name	Member Academic ID
1	Fedaa Alsabea	2230003675
2	Hessa Al-Turki	2230007134
3	Maha Al-Mansoori	2230002711
4	Sara Al-Idrissi	2230004327
5	Rimas Alajmi	2230002688
6	Fatima Al- Shihab	2230002074

Overview

In this assignment, you will perform Training Neural Net on the MNIST Dataset dataset using Pytorch. The goal is to explore the Basic Neural Network, understand its structure, and identify important issues such as architecture, learning rate, momentum and other parameters. You will also analyze relationships between number of layers and number of neurons for each layers to the overall Neural Network performance through their tuning hyperparameters.

Objective

Explore the dataset using PyTorch to understand:

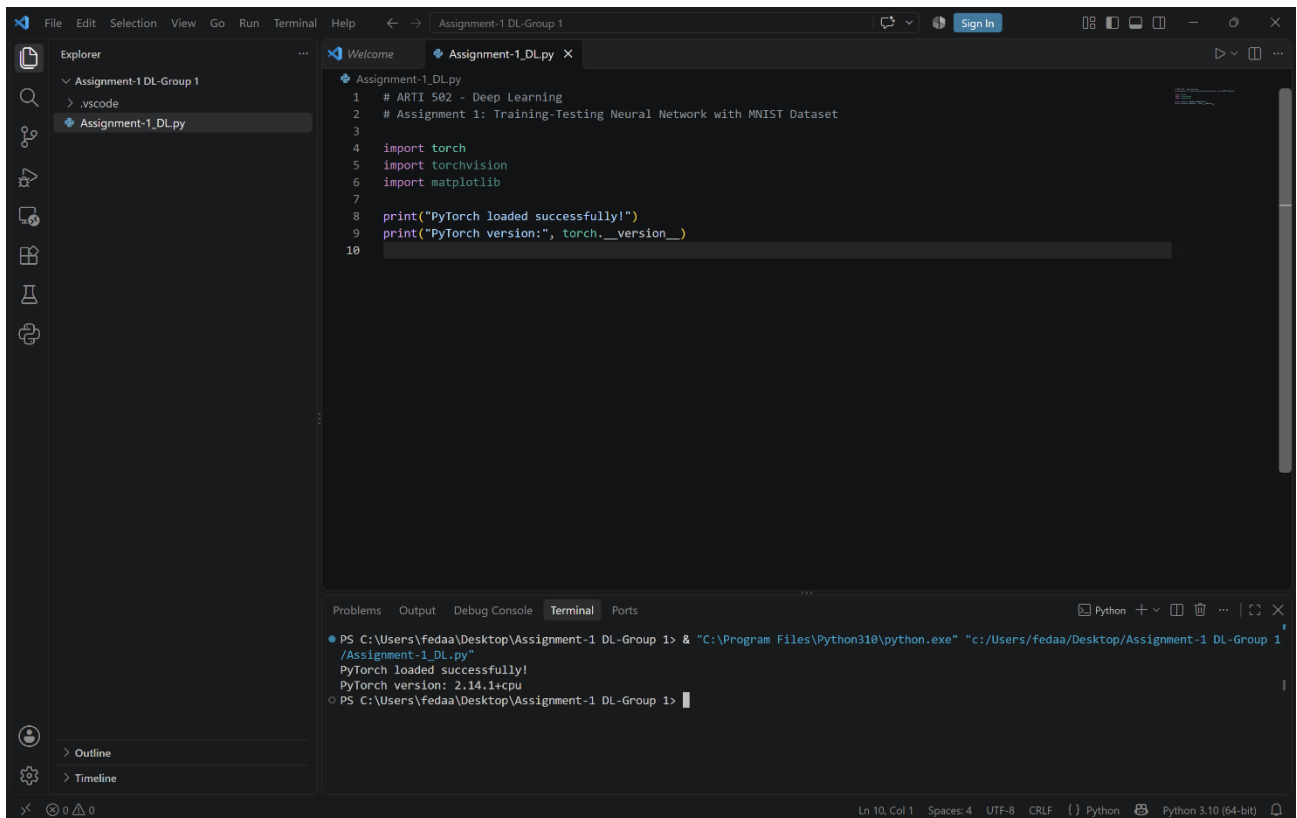
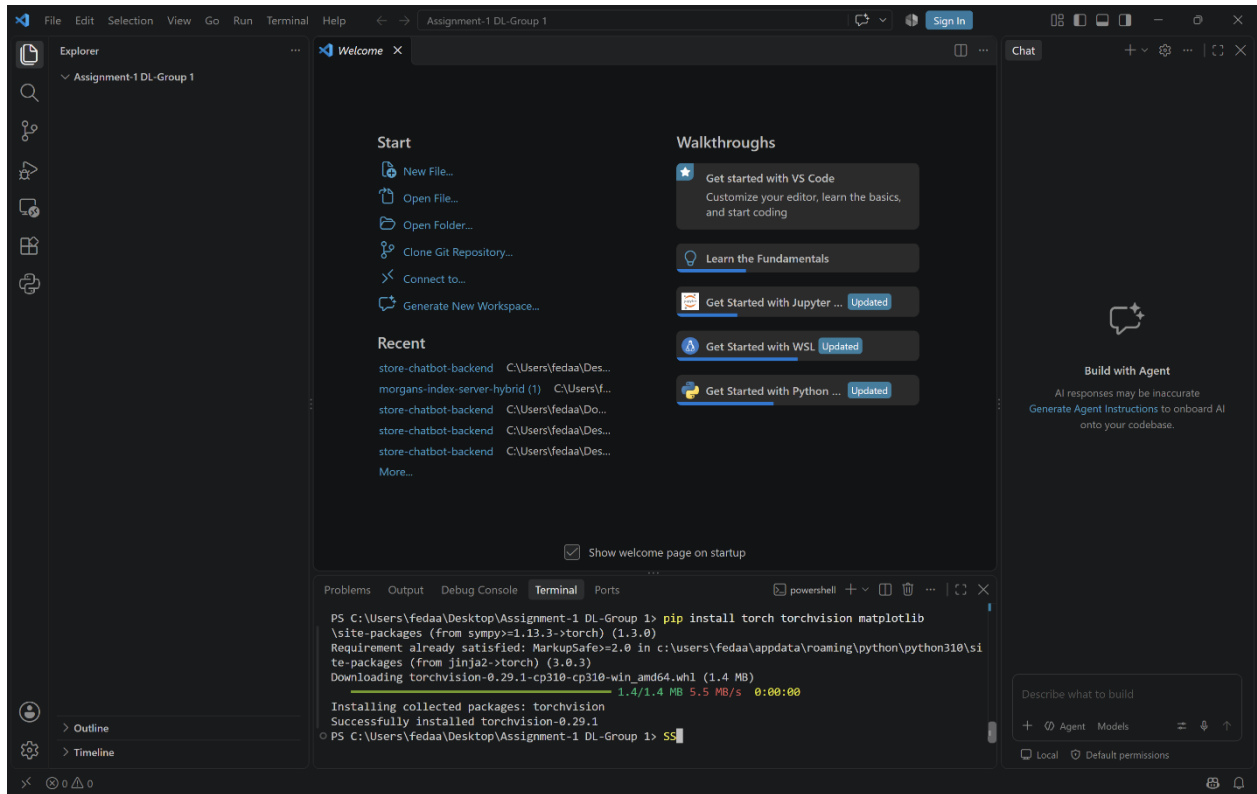
- Simple Neural Networks structure
- How to do Training-Testing Neural Networks
- How to evaluate the performance
- How to determine learning rate, momentum and model hyperparameters

Tasks (with PyTorch)

Task 1: Install PyTorch

Instructions:

- Install PyTorch and load it to your python code and save it as '**Assigment-1 DL**'
- Screenshoot how your load PyTorch to your python source code.



Task 2: Download and Load the Dataset

Instructions:

- PyTorch has some built tools for downloading and loading common datasets. You will be playing around with MNIST in this assignment. It is a dataset of 28x28 grayscale handwritten digits 0-9. There are 50,000 images in the training set and 10,000 in the test set.
- Download the dataset directly from your python code then load it into the the workspace.

Deliverable:

- Screenshot showing the dataset loaded successfully.

The screenshot shows a Visual Studio Code interface with two main panels. The top panel displays a Python file named `Assignment-1_DL.py`. The code imports `torchvision.datasets` and `torch.utils.data.DataLoader`, converts MNIST images to PyTorch tensors, downloads and loads training and testing datasets from a local directory (`./data`), and creates data loaders with a batch size of 32. The bottom panel shows the Terminal window with the command prompt at `C:\Users\Fedaa\Desktop\Assignment-1 DL-Group 1 >`. The output indicates that PyTorch was loaded successfully (version 2.14.1+cpu) and the MNIST dataset was loaded successfully. It also shows the number of training (60000) and testing (10000) images.

```

14 from torchvision import datasets, transforms
15 from torch.utils.data import DataLoader
16
17 # Convert MNIST images to PyTorch tensors
18 transform = transforms.ToTensor()
19
20 # Download and load the training dataset
21 train_dataset = datasets.MNIST(
22     root="./data",
23     train=True,
24     download=True,
25     transform=transform
26 )
27
28 # Download and load the testing dataset
29 test_dataset = datasets.MNIST(
30     root="./data",
31     train=False,
32     download=True,
33     transform=transform
34 )
35
36 # Create data loaders
37 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
38 test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

```

```

PS C:\Users\Fedaa\Desktop\Assignment-1 DL-Group 1> & "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1 /Assignment-1_DL.py"
PyTorch loaded successfully!
PyTorch version: 2.14.1+cpu
100% |██████████████████████████████████████████████████████████████████████████| 9.91M/9.91M [00:01<00:00, 5.81MB/s]
100% |██████████████████████████████████████████████████████████████████████████| 28.9k/28.9k [00:00<00:00, 182kB/s]
100% |██████████████████████████████████████████████████████████████████████████| 1.65M/1.65M [00:01<00:00, 1.52MB/s]
100% |██████████████████████████████████████████████████████████████████████████| 4.54k/4.54k [00:00<?, ?B/s]

MNIST dataset loaded successfully!
Training images: 60000
Testing images: 10000
PS C:\Users\Fedaa\Desktop\Assignment-1 DL-Group 1>

```

Task 3: Inspect the Dataset

Instructions:

- It's worthwhile to check out how the dataloader loads the images into tensors. You can print out the size of the loaded data in the images tensors. Also you can use matplotlib to see the visualization of the images data

Deliverable:

- Provide screenshots how to find:
 - Images tensor size
 - Data images vizualization using matplotlib

```

35
36 # Create data loaders
37 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
38 test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
39
40 print("\nMNIST dataset loaded successfully!")
41 print("Training images:", len(train_dataset))
42 print("Testing images:", len(test_dataset))
43
44
45
46
47 # Task 3: Inspect the Dataset
48
49 # Get one batch of images and labels
50 images, labels = next(iter(train_loader))
51
52 print("\n--- Task 3: Inspect the Dataset ---")
53 print("Images tensor size:", images.shape)
54 print("Labels tensor size:", labels.shape)

```

```

PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1/Assignment-1_DL.py"
PyTorch loaded successfully!
PyTorch version: 2.14.1+cpu

MNIST dataset loaded successfully!
Training Images: 60000
Testing Images: 10000

--- Task 3: Inspect the Dataset ---
Images tensor size: torch.Size([32, 1, 28, 28])
Labels tensor size: torch.Size([32])
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>

```



Figure 1

Sample Images from MNIST Dataset

Label: 5	Label: 5	Label: 6	Label: 1	Label: 4
Label: 9	Label: 3	Label: 3	Label: 4	Label: 1

```
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1/Assignment-1_DL.py"
PyTorch loaded successfully!
PyTorch version: 2.14.1+cpu

MNIST dataset loaded successfully!
Training images: 60000
Testing images: 10000

--- Task 3: Inspect the Dataset ---
Images tensor size: torch.Size([32, 1, 28, 28])
Labels tensor size: torch.Size([32])
```

Task 4: Creating A Simple Network

Instructions:

- Write your Python code to setup the following simple network architecture:

First you'll build a very basic neural network with a single hidden layer of neurons. This means one fully connected layer of weights connects the input to the hidden neurons and one fully connected layer connects the hidden neurons to the output. Use the RELU activation function on the hidden neuron values as our nonlinearity.

These fully connected (or `nn.Linear`) layers expect a 2D input tensor where that is $N \times I$ where N is the number of data points in a **mini batch** and I is the **number of inputs**. However, our data is formatted in $(N \times C \times H \times W)$ right now so we need to tell PyTorch to rearrange it using `torch.flatten`. C , H and W are refer to Color, Height and Width respectively.

After being **flattened** our data goes from $(32 \times 1 \times 28 \times 28)$ to being (32×784)

Deliverables:

- Screenshot of source code
- Verify your code with above requirement.

```

76 class SimpleNeuralNetwork(nn.Module):
77     def forward(self, x):
78         # Hidden layer with RELU activation
79         x = F.relu(self.fc1(x))
80
81         # Output layer
82         x = self.fc2(x)
83
84         return x
85
86 # Create and verify the network
87 model = SimpleNeuralNetwork()
88
89 print("\n--- Task 4: Simple Neural Network ---")
90 print(model)
91
92 # Pass one batch through the network
93 sample_output = model(images)
94
95 print("\nInput images shape:", images.shape)
96 print("Output shape:", sample_output.shape)

```

```

PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> & "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1
/Assignment-1_DL.py"
Images tensor size: torch.Size([32, 1, 28, 28])
Labels tensor size: torch.Size([32])

--- Task 4: Simple Neural Network ---
SimpleNeuralNetwork(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)

Input images shape: torch.Size([32, 1, 28, 28])
Output shape: torch.Size([32, 10])
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>

```

Task 5: Training Function

Instructions:

- Now for training the network. Your train function takes as input the net to train and the dataloader for the training data. It also takes some optional parameters to control training.
- For your network you will be using PyTorch's built in nn.CrossEntropyLoss. This will apply a softmax to our network's output, calculate the log-probability assigned to each class, then try to minimize the negative log likelihood of our data (AKA maximize the likelihood)
- For your optimizer you are using stochastic gradient descent with learning rate and momentum

Deliverables:

- Screenshot of your python source code that contains your training function with above requirements.

```

111
112 # Task 5: Training Function
113
114 import torch.optim as optim
115
116 def train_network(net, train_loader, epochs=5, learning_rate=0.01, momentum=0.9):
117
118     # Loss function
119     criterion = nn.CrossEntropyLoss()
120
121     # SGD optimizer with learning rate and momentum
122     optimizer = optim.SGD(
123         net.parameters(),
124         lr=learning_rate,
125         momentum=momentum
126     )
127
128     losses = []
129     accuracies = []
130
131     for epoch in range(epochs):
132         running_loss = 0.0
133         correct = 0
134         total = 0
135
136         net.train()

```

```

PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> & "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1
/Assignment-1_DL.py"
Images tensor size: torch.Size([32, 1, 28, 28])
Labels tensor size: torch.Size([32])

--- Task 4: Simple Neural Network ---
SimpleNeuralNetwork(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)

Input images shape: torch.Size([32, 1, 28, 28])
Output shape: torch.Size([32, 10])
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>

```



Task 6: Training Network

Instructions:

- You will instantiate a new network and train it on your training data.
- Your training function prints out some debug information about the epoch, batch number, and current loss and accuracy values. It also returns a list of all the losses on your mini-batches so you can plot them all once training has finished.

Deliverables:

- Screenshot of instantiate training source code
- Screenshot of training debug which include epoch, batch number and current loss, accuracy values and graph for overall processes (loss and accuracy for each epoch).

```

189
190
191 # Task 6: Training the Network
192
193 print("\n--- Task 6: Training the Network ---")
194
195 # Instantiate a new neural network
196 trained_model = SimpleNeuralNetwork()
197
198 # Train the network
199 training_losses, training_accuracies, batch_losses = train_network(
200     trained_model,
201     train_loader,
202     epochs=5,
203     learning_rate=0.01,
204     momentum=0.9
205 )
206
207 # Print proof that the loss of every mini-batch was collected
208 print(f"Mini-batch loss values collected: {len(batch_losses)}")
209
210 # Plot training Loss and Accuracy
211
212 epochs_range = range(1, len(training_losses) + 1)
213
214 # Loss epoch

```

```

PS C:\Users\Fedaa\Desktop\Assignment-1 DL-Group 1> & "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1
/Assignment-1_DL.py"
Epoch [3/5], Batch [1000/1875], Current Loss: 0.0206, Accuracy: 96.57%
Epoch [3/5], Batch [1500/1875], Current Loss: 0.1711, Accuracy: 96.64%
Epoch [3/5] Completed - Loss: 0.1135, Accuracy: 96.70%
Epoch [4/5], Batch [500/1875], Current Loss: 0.8549, Accuracy: 97.21%
Epoch [4/5], Batch [1000/1875], Current Loss: 0.0390, Accuracy: 97.33%
Epoch [4/5], Batch [1500/1875], Current Loss: 0.0625, Accuracy: 97.34%
Epoch [4/5] Completed - Loss: 0.0895, Accuracy: 97.40%
Epoch [5/5], Batch [500/1875], Current Loss: 0.1349, Accuracy: 97.91%
Epoch [5/5], Batch [1000/1875], Current Loss: 0.0197, Accuracy: 97.78%
Epoch [5/5], Batch [1500/1875], Current Loss: 0.0329, Accuracy: 97.81%
Epoch [5/5] Completed - Loss: 0.0729, Accuracy: 97.78%
Mini-batch loss values collected: 9375

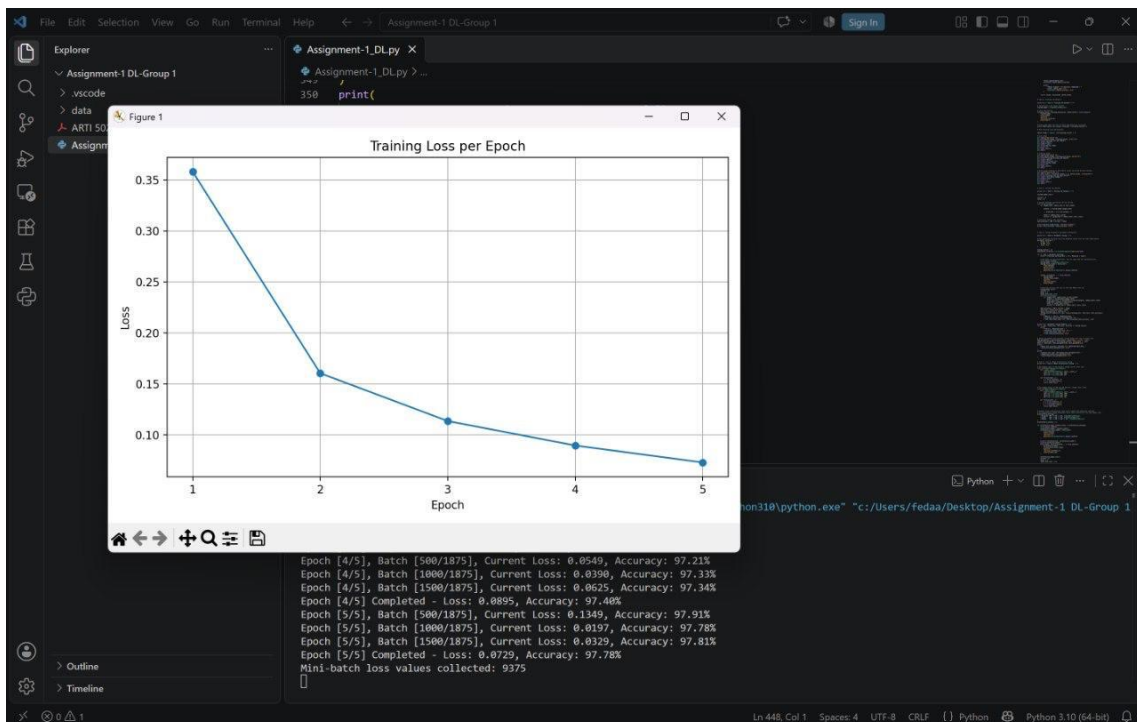
```

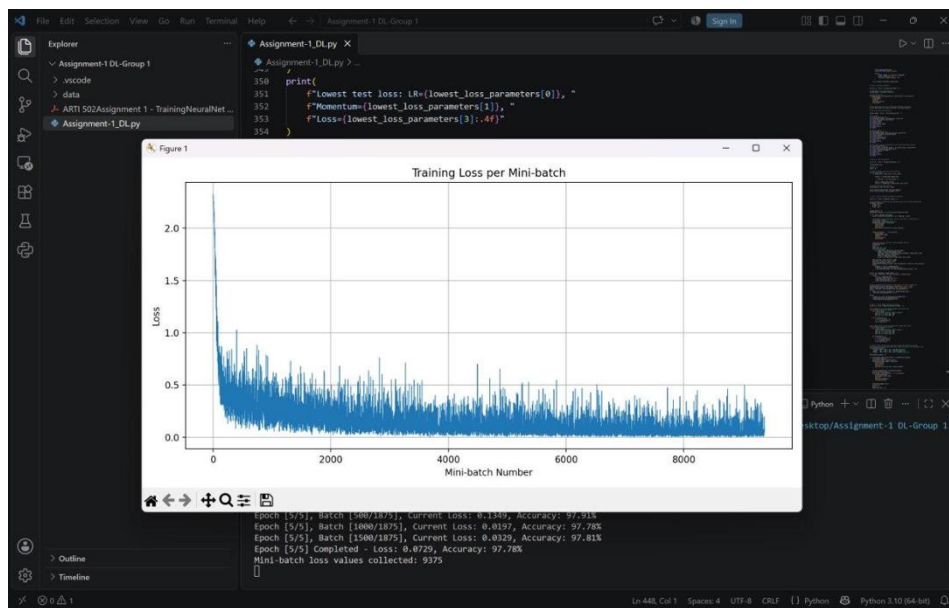
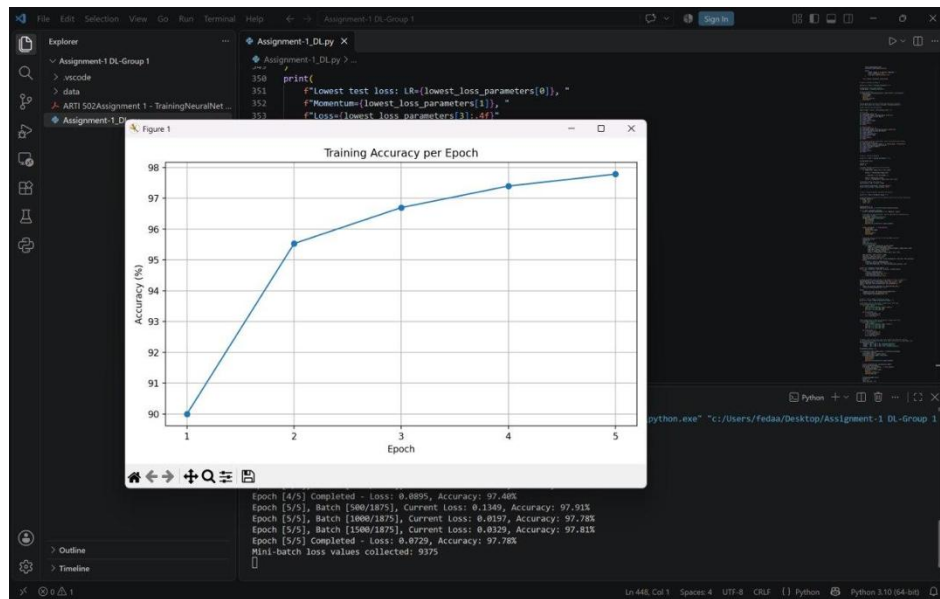


```

File Edit Selection View Go Run Terminal Help Assignment-1 DL-Group 1
Assignment-1.DLpy X
208 print(f"Mini-batch loss values collected: {len(batch_losses)}")
209
210 # Plot training Loss and Accuracy
211
212 epochs_range = range(1, len(training_losses) + 1)
213
214 # Loss graph
215 plt.figure(figsize=(8, 5))
216 plt.plot(epochs_range, training_losses, marker="o")
217 plt.title("Training Loss per Epoch")
218 plt.xlabel("Epoch")
219 plt.ylabel("Loss")
220 plt.xticks(epochs_range)
221 plt.grid(True)
222 plt.tight_layout()
223 plt.show()
224
225 # Accuracy graph
226 plt.figure(figsize=(8, 5))
227 plt.plot(epochs_range, training_accuracies, marker="o")
228 plt.title("Training Accuracy per Epoch")
229 plt.xlabel("Epoch")
230 plt.ylabel("Accuracy (%)")
231 plt.xticks(epochs_range)
232 plt.grid(True)
233 plt.tight_layout()
234 plt.show()
235
236 # Extra graph showing all mini-batch losses collected during training
237 plt.figure(figsize=(10, 5))
238 plt.plot(range(1, len(batch_losses) + 1), batch_losses, linewidth=0.7)
239 plt.title("Training Loss per Mini-batch")
240 plt.xlabel("Mini-batch Number")
241 plt.ylabel("Loss")
242 plt.grid(True)
243 plt.tight_layout()
244 plt.show()
245
Problems Output Debug Console Terminal Ports Jupyter
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>
Ln 448, Col 1 Spaces 4 UTF-8 CRLF Python Python 3.10 (64-bit)

```





The network was trained for 5 epochs. The training loss decreased to 0.0729, while the training accuracy increased to 97.78%. The mini-batch loss graph also shows an overall decreasing trend, indicating that the network learned successfully.

Task 7: Testing Network

Instructions:

- You can test your network on a few of your test images from task 2 and see what happens.

Deliverables:

- Screenshot your network testing source code
- Screenshot the results of the network testing

The screenshot shows a VS Code editor with a file named 'Assignment-1_DL.py'. The script is a Python program for testing a trained model. It includes comments in Arabic and English, and uses the 'torch' library for tensor operations. The script calculates the overall test accuracy by iterating through a test loader, comparing predicted labels with actual labels, and summing the correct predictions. The output is printed to the console.

```

252 # task 7: testing the Network
253
254 print("\n--- Task 7: Testing the Network ---")
255
256 trained_model.eval()
257
258 correct = 0
259 total = 0
260
261 # Disable gradient calculation during testing
262 with torch.no_grad():
263     for images_test, labels_test in test_loader:
264
265         outputs = trained_model(images_test)
266
267         _, predicted = torch.max(outputs, 1)
268
269         total += labels_test.size(0)
270         correct += (predicted == labels_test).sum().item()
271
272 # Calculate overall test accuracy
273 test_accuracy = 100 * correct / total
274
275 print(f"Correct predictions: {correct}/{total}")
276 print(f"Test Accuracy: {test_accuracy:.2f}%")
  
```

The terminal output shows the execution of the script. It displays the progress of training (Epochs 4 and 5) and then the results of the network testing. The test accuracy is 97.18%.

```

PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1> "C:\Program Files\Python310\python.exe" "c:\Users\fedaa\Desktop\Assignment-1 DL-Group 1\Assignment-1_DL.py"
Epoch [4/5], Batch [1000/1875], Current Loss: 0.0741, Accuracy: 97.28%
Epoch [4/5], Batch [1500/1875], Current Loss: 0.0593, Accuracy: 97.28%
Epoch [4/5] Completed - Loss: 0.0915, Accuracy: 97.34%
Epoch [5/5], Batch [500/1875], Current Loss: 0.0501, Accuracy: 97.90%
Epoch [5/5], Batch [1000/1875], Current Loss: 0.0840, Accuracy: 97.87%
Epoch [5/5], Batch [1500/1875], Current Loss: 0.2307, Accuracy: 97.86%
Epoch [5/5] Completed - Loss: 0.0748, Accuracy: 97.84%

--- Task 7: Testing the Network ---
Correct predictions: 9718/10000
Test Accuracy: 97.18%
PS C:\Users\fedaa\Desktop\Assignment-1 DL-Group 1>
  
```

Task 8: Tuning the parameters and model architecture

Instructions:

- Tune the parameters with different values of learning rate and momentum and write your best results (lowest loss and highest accuracy).
- Tune the architecture model with different values of number of layer and number of neuron and write your best results (lowest loss and highest accuracy).

Deliverables:

- Screen your source python code with the best results.

Parameter tuning

```

274
275 # Task 8: Tuning Parameters and Model Architecture
276
277 print("\n--- Task 8: Parameter Tuning ---")
278
279 # Test different learning rates and momentum values with the same model/epochs.
280 parameter_settings = [
281     (0.01, 0.9),
282     (0.001, 0.9),
283     (0.01, 0.5)
284 ]
285
286 tuning_results = []
287 evaluation_criterion = nn.CrossEntropyLoss(reduction="sum")
288
289 for lr, mom in parameter_settings:
290     print(f"\nTesting Learning Rate = {lr}, Momentum = {mom}")
291
292     # New model for each experiment. Use the same seed for reproducibility.
293     torch.manual_seed(42)
294     tuning_model = SimpleNeuralNetwork()
295     tuning_train_loader = DataLoader(
296         train_dataset,
297         batch_size=32,
298         shuffle=True,

```

```

PS C:\Users\Fedaa\Desktop\Assignment-1 DL-Group 1> & "C:\Program Files\Python310\python.exe" "c:/Users/Fedaa/Desktop/Assignment-1 DL-Group 1
/Assignment-1 DL.py"
Epoch [3/3], Batch [1000/1875], Current Loss: 0.5277, Accuracy: 97.49%
Epoch [3/3], Batch [1500/1875], Current Loss: 0.0204, Accuracy: 97.52%
Epoch [3/3] Completed - Loss: 0.0820, Accuracy: 97.56%

--- Architecture Tuning Summary ---
All models: epochs=3, LR=0.01, Momentum=0.9
Original 784 -> 128 -> 10 | Training Loss=0.1138 | Test Loss=0.1150 | Test Accuracy=96.66%
Wider 784 -> 256 -> 10 | Training Loss=0.1075 | Test Loss=0.1031 | Test Accuracy=96.99%
Deeper 784 -> 256 -> 128 -> 10 | Training Loss=0.0820 | Test Loss=0.0822 | Test Accuracy=97.36%
Best test accuracy architecture: Deeper 784 -> 256 -> 128 -> 10 (97.36%)
Lowest test loss architecture: Deeper 784 -> 256 -> 128 -> 10 (0.0822)
PS C:\Users\Fedaa\Desktop\Assignment-1 DL-Group 1>

```



```

Assignment-1.DL.py X
...
285 tuning_results = []
286 evaluation_criterion = nn.CrossEntropyLoss(reduction="sum")
287
288 for lr, mom in parameter_settings:
289     print(f"\nTesting Learning Rate = {lr}, Momentum = {mom}")
290
291     # New model for each experiment. Use the same seed for reproducibility.
292     torch.manual_seed(42)
293     tuning_model = SimpleNeuralNetwork()
294     tuning_train_loader = DataLoader(
295         train_dataset,
296         batch_size=32,
297         shuffle=True,
298         generator=torch.Generator().manual_seed(42)
299     )
300
301     losses, accuracies, _ = train_network(
302         tuning_model,
303         tuning_train_loader,
304         epochs=3,
305         learning_rate=lr,
306         momentum=mom
307     )
308
309     # Evaluate accuracy and loss on the same MNIST test set.
310     tuning_model.eval()
311     correct = 0
312     total = 0
313     total_test_loss = 0.0
314
315     with torch.no_grad():
316         for images_test, labels_test in test_loader:
317             outputs = tuning_model(images_test)
318             total_test_loss += evaluation_criterion(outputs, labels_test).item()
319             predicted = outputs.argmax(dim=1)
320             total += labels_test.size(0)
321             correct += (predicted == labels_test).sum().item()
322

```

```

...
322
323 test_accuracy = 100 * correct / total
324 test_loss = total_test_loss / total
325 final_training_loss = losses[-1]
326 tuning_results.append((lr, mom, final_training_loss, test_loss, test_accuracy))
327 print(
328     f"Result -> LR={lr}, Momentum={mom}, "
329     f"Training Loss={final_training_loss:.4f}, "
330     f"Test Loss={test_loss:.4f}, Test Accuracy={test_accuracy:.2f}%"
331 )
332
333 print("\n--- Parameter Tuning Summary ---")
334 for lr, mom, train_loss, test_loss, accuracy in tuning_results:
335     print(
336         f"LR={lr}, Momentum={mom} | "
337         f"Training Loss={train_loss:.4f} | "
338         f"Test Loss={test_loss:.4f} | "
339         f"Test Accuracy={accuracy:.2f}%"
340     )
341
342 # Select by highest test accuracy, using lowest test loss to break ties.
343 best_parameters = max(tuning_results, key=lambda r: (r[4], -r[3]))
344 lowest_loss_parameters = min(tuning_results, key=lambda r: r[3])
345 best_lr, best_mom = best_parameters[0], best_parameters[1]
346 print(
347     f"Best test accuracy: LR={best_lr}, Momentum={best_mom}, "
348     f"Accuracy={best_parameters[4]:.2f}%"
349 )
350 print(
351     f"Lowest test loss: LR={lowest_loss_parameters[0]}, "
352     f"Momentum={lowest_loss_parameters[1]}, "
353     f"Loss={lowest_loss_parameters[3]:.4f}"
354 )
355
356 # Task 8 - Part 2: Model Architecture Tuning
357 print("\n--- Task 8: Model Architecture Tuning ---")
358

```

```

Problems Output Debug Console Terminal Ports Jupyter
Python Python 3.10 (64-bit)

PS C:\Users\Fedaa\Desktop\Assignment-1 DL-Group 1> & "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1/Assignment-1.DL.py"
Epoch [3/3] Completed - Loss: 0.2619, Accuracy: 92.61%
Result -> LR=0.01, Momentum=0.5, Training Loss=0.2619, Test Loss=0.2406, Test Accuracy=92.94%

--- Parameter Tuning Summary ---
LR=0.01, Momentum=0.9 | Training Loss=0.1138 | Test Loss=0.1150 | Test Accuracy=96.66%
LR=0.001, Momentum=0.9 | Training Loss=0.3227 | Test Loss=0.2953 | Test Accuracy=91.52%
LR=0.01, Momentum=0.5 | Training Loss=0.2619 | Test Loss=0.2406 | Test Accuracy=92.94%
Best test accuracy: LR=0.01, Momentum=0.9, Accuracy=96.66%
Lowest test loss: LR=0.01, Momentum=0.9, Loss=0.1150

--- Task 8: Model Architecture Tuning ---

```



Architecture tuning

```

File Edit Selection View Go Run Terminal Help Assignment-1 DL-Group 1
Assignment-1_DL.py X
Assignment-1_DL.py > ...
356
357 # Task 8 - Part 2: Model Architecture Tuning
358 print("\n--- Task 8: Model Architecture Tuning ---")
359
360 # One hidden layer of 256 neurons: change neuron count only.
361 class WiderNeuralNetwork(nn.Module):
362     def __init__(self):
363         super(WiderNeuralNetwork, self).__init__()
364         self.fc1 = nn.Linear(784, 256)
365         self.fc2 = nn.Linear(256, 10)
366
367     def forward(self, x):
368         x = torch.flatten(x, 1)
369         x = F.relu(self.fc1(x))
370         return self.fc2(x)
371
372
373 # Two hidden layers of 256 and 128 neurons: change layer count.
374 class TunedNeuralNetwork(nn.Module):
375     def __init__(self):
376         super(TunedNeuralNetwork, self).__init__()
377         self.fc1 = nn.Linear(784, 256)
378         self.fc2 = nn.Linear(256, 128)
379         self.fc3 = nn.Linear(128, 10)
380
381     def forward(self, x):
382         x = torch.flatten(x, 1)
383         x = F.relu(self.fc1(x))
384         x = F.relu(self.fc2(x))
385         return self.fc3(x)
386
387
388 # Compare three architectures, each with 3 epochs and identical settings.
389 # An independently seeded Dataloader gives each architecture the same image order.
390 architecture_settings = [
391     ("Original 784 -> 128 -> 10", SimpleNeuralNetwork),
392     ("Wider 784 -> 256 -> 10", WiderNeuralNetwork),
393     ("Deeper 784 -> 256 -> 128 -> 10", TunedNeuralNetwork)
394 ]

```

```

File Edit Selection View Go Run Terminal Help Assignment-1 DL-Group 1
Assignment-1_DL.py X
Assignment-1_DL.py > ...
394 }
395 architecture_results = []
396
397 for architecture_name, network_class in architecture_settings:
398     torch.manual_seed(42)
399     architecture_model = network_class()
400     architecture_train_loader = DataLoader(
401         train_dataset,
402         batch_size=32,
403         shuffle=True,
404         generator=torch.Generator().manual_seed(42)
405     )
406
407     print(f"\nArchitecture: {architecture_name}")
408     print(architecture_model)
409     arch_losses, arch_accuracies, _ = train_network(
410         architecture_model,
411         architecture_train_loader,
412         epochs=3,
413         learning_rate=best_lr,
414         momentum=best_mom
415     )
416
417     architecture_model.eval()
418     correct = 0
419     total = 0
420     total_test_loss = 0.0
421     with torch.no_grad():
422         for images_test, labels_test in test_loader:
423             outputs = architecture_model(images_test)
424             total_test_loss += evaluation_criterion(outputs, labels_test).item()
425             predicted = outputs.argmax(dim=1)
426             total += labels_test.size(0)
427             correct += (predicted == labels_test).sum().item()
428
429     test_accuracy = 100 * correct / total
430     test_loss = total_test_loss / total
431     final_train_loss = arch_losses[-1]

```



```

441 architecture_train_loader,
442 epochs=3,
443 learning_rate=best_lr,
444 momentum=best_mom
445 )
446
447 architecture_model.eval()
448 correct = 0
449 total = 0
450 total_test_loss = 0.0
451 with torch.no_grad():
452     for images_test, labels_test in test_loader:
453         outputs = architecture_model(images_test)
454         total_test_loss += evaluation_criterion(outputs, labels_test).item()
455         predicted = outputs.argmax(dim=1)
456         total += labels_test.size(0)
457         correct += (predicted == labels_test).sum().item()
458
459 test_accuracy = 100 * correct / total
460 test_loss = total_test_loss / total
461 final_training_loss = arch_losses[-1]
462 architecture_results.append(
463     (architecture_name, final_training_loss, test_loss, test_accuracy)
464 )
465
466 print("\n--- Architecture Tuning Summary ---")
467 print(f"All models: epochs=3, LR={best_lr}, Momentum={best_mom}")
468 for name, train_loss, test_loss, accuracy in architecture_results:
469     print(
470         f"{name} | Training Loss={train_loss:.4f} | "
471         f"Test Loss={test_loss:.4f} | Test Accuracy={accuracy:.2f}%"
472     )
473
474 best_architecture = max(architecture_results, key=lambda r: (r[3], -r[2]))
475 lowest_loss_architecture = min(architecture_results, key=lambda r: r[2])
476 print(f"Best test accuracy architecture: {best_architecture[0]} ({best_architecture[3]:.2f}%)"
477 print(f"Lowest test loss architecture: {lowest_loss_architecture[0]} ({lowest_loss_architecture[2]:.4f})")
478

```

```

PS C:\Users\Fedaa\Desktop\Assignment-1 DL-Group 1> "C:\Program Files\Python310\python.exe" "c:/Users/fedaa/Desktop/Assignment-1 DL-Group 1 /Assignment-1_DL.py"
Epoch [3/3], Batch [1000/1875], Current Loss: 0.5277, Accuracy: 97.49%
Epoch [3/3], Batch [1500/1875], Current Loss: 0.0204, Accuracy: 97.52%
Epoch [3/3] Completed - Loss: 0.0828, Accuracy: 97.56%

--- Architecture Tuning Summary ---
All models: epochs=3, LR=0.01, Momentum=0.9
Original 784 -> 128 -> 10 | Training Loss=0.1138 | Test Loss=0.1150 | Test Accuracy=96.66%
Wider 784 -> 256 -> 10 | Training Loss=0.1075 | Test Loss=0.1031 | Test Accuracy=96.99%
Deeper 784 -> 256 -> 128 -> 10 | Training Loss=0.0828 | Test Loss=0.0822 | Test Accuracy=97.36%
Best test accuracy architecture: Deeper 784 -> 256 -> 128 -> 10 (97.36%)
Lowest test loss architecture: Deeper 784 -> 256 -> 128 -> 10 (0.0822)
PS C:\Users\Fedaa\Desktop\Assignment-1 DL-Group 1>

```

We tested different learning rates, momentum values, and network architectures. The best parameter combination was a learning rate of 0.01 and momentum of 0.9, achieving 96.66% test accuracy and 0.1150 test loss. Among the three architectures, the deeper network achieved the best performance, with 97.36% test accuracy and 0.0822 test loss.

Task 9: Save your source code

Instructions:

- Save your python source code

Deliverable:

- Submit your Github link of your source code along with this document, which includes all screenshots via Blackboard.

Github link:

<https://github.com/fedaaalsabea/Deep-Learning-Assignment-1-Group1>

- End of Assignment -

Grading Rubric

Criterion	Excellent	Good	Developing	Poor	Marks
Task 1: Install PyTorch	PyTorch installed successfully and PyTorch loaded correctly with clear structure	Installed and PyTorch loaded with minor issues	pyTorch exists but unclear or incomplete	Not completed / missing	0.5
Task 2: Download and Load the Dataset	Dataset loaded correctly with clear screenshot showing successful execution	Loaded correctly but screenshot is unclear	Dataset loaded with issues or incomplete evidence	Not loaded / missing screenshot	0.5
Task 3: Inspect the Dataset	All clear screenshots	Minor mistakes or missing small details	Several missing description or unclear screenshots	Not answered / incorrect / missing evidence	0.5
Task 4: Creating A Simple Network	A simple network correctly created, screenshot included	Minor reporting issues	Node used but missing result explanation or screenshot	Not done	2
Task 6: Training Function	Training function configured correctly with clear screenshots of output	Minor reporting issues	Training Function used but missing result explanation or screenshot	Not done	1
Task 7: Testing Network	Correct Testing network created, screenshot included.	Testing network correct but result description is weak	Testing network exists but unclear results description	Not done	1
Task 8: Tuning the parameters and model architecture	Tuning parameters and model architecture created, screenshot included, and results description	Tuning Parameters and model architecture correct but results description is weak	Tuning Parameters and model architecture correct but incorrect results description	Not done	1.5



	explained accurately				
Task 9: Save your source code	Source code save succefully	Source code save succefully done but minor missing item	Source code save incomplete or file format incorrect	Not submitted / missing	0.5
Total					7.5